فصل ۲: شرط، switch، حلقه، break و continue

چرا این فصل؟ تست‌های «خروجی کد» تقریباً همیشه یک حلقه یا switch دارند. تله‌ها محدود و تکراری‌اند: else آویزان، سقوط در switch، `;` بعد از for یا if، و شمردن تکرارها. **اعتبار:** خروجی همهٔ کدها با `gcc` اجرا و چک شده است. **زمان مطالعه:** یک بلوک ۹۰ دقیقه‌ای.

درس ۱: if و else

سه قاعده

۱. **هر عدد غیرصفر «درست» است**، حتی عدد منفی. فقط صفر «نادرست» است.

۲. **else آویزان:** else به **نزدیک‌ترین if بدون else** می‌چسبد، نه به if ای که تورفتگی نشان می‌دهد. تورفتگی برای کامپایلر مهم نیست.

۳. **`;` بعد از if** یعنی بدنهٔ if خالی است. دستور بعدی **همیشه** اجرا می‌شود.

مثال (else آویزان):

```
int a = 0, b = 1;
if (a)
    if (b) printf("X");
else printf("Y");      // این else مال if (b) است!
printf("Z");
```

خروجی فقط `Z` است. چون a صفر است، کل `if (b) … else …` اجرا نمی‌شود.

تمرین درس ۱

```
// 1
int x = 3;  if (x > 5);  printf("A");
// 2
int y = 0;  if (y = 5) printf("%d", y);
// 3
int z = -2;  printf("%s", z ? "true" : "false");
```

🔑 **جواب‌ها:** پاسخ‌نامهٔ انتهای فصل، بخش ۱ (درس ۱: if و else)

درس ۲: switch

قواعد

- مقدار switch و برچسب‌های case باید **صحیح یا کاراکتر** و **ثابت** باشند. بازه (مثل `case 1..5` در C) استاندارد نیست.
- **سقوط (fall-through):** اگر case با `break` تمام نشود، اجرا به case بعدی **ادامه می‌یابد** و دیگر شرط آن چک نمی‌شود.
- `default` می‌تواند هر جایی باشد. فقط وقتی اجرا می‌شود که هیچ case ای جور نشود، و بعد از آن هم سقوط ادامه دارد.

مثال:

```
int k = 2, s = 0;
switch (k) {
    case 1: s += 1;
    case 2: s += 2;          // از اینجا وارد می‌شویم
    case 3: s += 3; break;   // سقوط تا اینجا
    case 4: s += 4;
    default: s += 10;
}
```

`s = 2 + 3 = 5`. با `k = 4` می‌شد `4 + 10 = 14`.

تمرین درس ۲

```
int k = 7, s = 0;
switch (k) {
    default: s += 10;
    case 1:  s += 1; break;
    case 2:  s += 2;
}
printf("%d", s);
```

> 🔑 **جواب:** پاسخ‌نامهٔ انتهای فصل، بخش ۲ (درس ۲: switch)

درس ۳: حلقه‌ها

حلقه	نکته
`for (init; cond; step)`	بعد از هر تکرار `step` اجرا می‌شود و بعد `cond` چک می‌شود
`while (cond)`	ممکن است **صفر بار** اجرا شود
`do { … } while (cond);`	**حداقل یک بار** اجرا می‌شود

تله‌ها

۱. **`;` بعد از for:** `for (i = 0; i < 5; i++);` حلقه‌ای با بدنهٔ خالی است. بعد از آن `i == 5` است.

۲. **شمردن تکرار:** تعداد تکرارهای `for (i = a; i <= b; i += d)` برابر `⌊(b−a)/d⌋ + 1` است.

۳. **پسوندی در شرط:** در `while (i++ < 3)` مقایسه با مقدار **قدیمی** i انجام می‌شود، ولی بدنه مقدار **جدید** را می‌بیند.

۴. **کاما در for:** `for (i = 0, j = 10; i < j; i += 2, j -= 3)` دو متغیر را هم‌زمان جلو می‌برد.

۵. **unsigned:** شرط `u >= 0` برای unsigned همیشه درست است و حلقه بی‌نهایت می‌شود.

تمرین درس ۳

```
// 1
int i = 10;  do { printf("%d ", i); i++; } while (i < 5);
// 2
int c = 0;  for (i = 1; i <= 100; i += 3) c++;   printf("%d %d", c, i);
// 3
i = 0;  while (i++ < 3) printf("%d ", i);   printf("| %d", i);
// 4
i = 5;  while (i --> 0) printf("%d", i);       // یعنی (i--) > 0
// 5
int c2 = 0;  for (int a = 0; a < 4; a++) for (int b = a; b < 4; b++) c2++;
```

> 🔑 **جواب‌ها:** پاسخ‌نامهٔ انتهای فصل، بخش ۳ (درس ۳: حلقه‌ها)

درس ۴: break و continue

دستور	کار
`break`	فقط از **درونی‌ترین** حلقه یا switch بیرون می‌رود

دستور	کار
`continue`	بقیهٔ بدنه را رد می‌کند. در `for` بعدش **step** اجرا می‌شود؛ در `while` مستقیم سراغ شرط می‌رود

⚠️ در `while` اگر `i++` بعد از `continue` باشد، ممکن است اجرا نشود و حلقه بی‌نهایت شود. در `for` این مشکل نیست، چون step در خود for است.

مثال:

```
int s = 0;
for (int i = 1; i <= 10; i++) {
    if (i % 2 == 0) continue;   // زوج‌ها رد می‌شوند
    if (i > 7) break;           // با 9 خارج می‌شویم
    s += i;
}
```

`s = 1 + 3 + 5 + 7 = 16`.

تمرین درس ۴

```
// 1
int c = 0;
for (int i = 0; i < 3; i++)
    for (int j = 0; j < 3; j++) { if (j == 1) break; c++; }
// 2
int s = 0, i = 0;
while (i < 10) { i++; if (i % 3 == 0) continue; s += i; }
```

🔑 **جواب‌ها:** پاسخ‌نامهٔ انتهای فصل، بخش ۴ (درس ۴: break و continue)

📝 آزمون فصل ۲

این درس در کنکور ارشد کامپیوتر سؤال سابقه ندارد. پس این ۱۰ تست **تألیفی** هستند و خروجی همه با `gcc` تأیید شده است.

زمان: ۱۵ دقیقه. خروجی هر کد چیست؟

۱.

```
int x = 5, y = 10;
if (x > y)
    if (y > 0) x = 1;
else x = 2;
printf("%d", x);
```

۱) `5` ۲) `1` ۳) `2` ۴) خطای کامپایل

۲.

```
int n = 3, r = 0;
switch (n) {
    case 1: r++;
    case 3: r += 3;
    case 5: r += 5;
    default: r += 100;
}
printf("%d", r);
```

۱) `3` ۲) `8` ۳) `108` ۴) `103`

۳.

```
int i = 0, s = 0;
do { s += i; i += 2; } while (i < 0);
printf("%d %d", s, i);
```

۱) `0 0` ۲) `0 2` ۳) `2 2` ۴) حلقه اجرا نمی‌شود و `0 0`

۴.

```
int c = 0;
for (int i = 1; i <= 20; i *= 3) c++;
printf("%d", c);
```

۱) `2` ۲) `3` ۳) `4` ۴) `7`

۵.

```
int s = 0;
for (int i = 0; i < 10; i++) {
    if (i == 2) continue;
    if (i == 6) break;
    s += i;
}
printf("%d", s);
```

۱) 15 ۲) 13 ۳) 19 ۴) 43

۶.

```
int i = 10;
while (i-- > 7);
printf("%d", i);
```

۱) 7 ۲) 6 ۳) 9 ۴) حلقهٔ بی‌نهایت

۷.

```
int c = 0;
for (int i = 0; i < 5; i++)
    for (int j = 0; j < i; j++)
        if ((i + j) % 2) c++;
printf("%d", c);
```

۱) 4 ۲) 5 ۳) 6 ۴) 10

۸.

```
int x = -4, r = 0;
switch (x % 3) {
    case 1:  r = 1; break;
    case 2:  r = 2; break;
    case -1: r = 3; break;
    default: r = 4;
}
printf("%d", r);
```

۱) 1 ۲) 2 ۳) 3 ۴) 4

۹.

```
int i, j, c = 0;
for (i = 0, j = 10; i < j; i += 2, j -= 2) c++;
printf("%d %d %d", c, i, j);
```

۱) 4 3 6 ۲) 6 4 2 ۳) 6 4 3 ۴) 0 10 5

۱۰.

```
int n = 0;
for (int i = 1; i <= 3; i++)
    for (int j = 1; j <= 3; j++) {
        if (j == i) continue;
        if (j > i) break;
        n += 10 * i + j;
    }
printf("%d", n);
```

۱) `84` ۲) `63` ۳) `21` ۴) `117`

> 🔑 **جواب‌ها و راه‌حل:** پاسخ‌نامهٔ انتهای فصل، بخش ۵ (آزمون فصل ۲)

🗂 کارت خلاصهٔ فصل ۲

- غیرصفر یعنی درست. else به نزدیک‌ترین if بدون else می‌چسبد. `;` بعد از if یا for یعنی بدنهٔ خالی.
- switch بدون break سقوط می‌کند. default هر جا باشد، بعدش هم سقوط ادامه دارد. علامت `%` مهم است.
- do-while حداقل یک بار اجرا می‌شود. تعداد تکرار `⌊(b−a)/d⌋ + 1`. `x++ < n` با مقدار قدیمی مقایسه می‌کند.
- break فقط درونی‌ترین حلقه را می‌شکند. continue در for به step می‌رود و در while به شرط.

🔑 پاسخ‌نامهٔ فصل

بخش ۱: درس ۱: if و else

۱. `A` (بدنهٔ if خالی است)

۲. `5` (انتساب، مقدار ۵ دارد که درست است)

۳. `true` (منفی هم غیرصفر است)

بخش ۲: درس ۲: switch

۱. `11` . هیچ case ای جور نیست، پس default اجرا می‌شود. چون break ندارد، به `case 1` سقوط می‌کند و آنجا break می‌خورد.

بخش ۳: درس ۳: حلقه‌ها

۱. `10` (یک بار اجرا می‌شود، با اینکه شرط از اول نادرست است)

۲. `34 103` (`[99/3] + 1 = 34`؛ آخرین مقداری که شرط را نقض کرد ۱۰۳ است)

۳. `1 2 3 | 4`

۴. `43210`

۵. `10` (`1 + 2 + 3 + 4`)

بخش ۴: درس ۴: break و continue

۱. `3` . break فقط حلقهٔ درونی را می‌شکند، پس در هر دور بیرونی فقط `j = 0` شمرده می‌شود.

۲. `37` . جمع ۱ تا ۱۰ منهای مضرب‌های ۳: `55 − (3+6+9) = 37` . چون `i++` قبل از continue است، حلقه بی‌نهایت نمی‌شود.

بخش ۵: آزمون فصل ۲

سؤال	۱	۲	۳	۴	۵	۶	۷	۸	۹	۱۰
جواب	۱	۳	۲	۲	۲	۲	۳	۳	۱	۱

۱. else مال `if (y > 0)` است. شرط بیرونی (`5 > 10`) نادرست است، پس هیچ‌چیز اجرا نمی‌شود و x همان **۵** می‌ماند.

۲. از `case 3` وارد می‌شود و تا آخر سقوط می‌کند: `3 + 5 + 100 = 108`.

۳. do-while یک بار اجرا می‌شود: `s = 0` و `i = 2`. بعد شرط `2 < 0` نادرست است.

۴. i مقدارهای ۱، ۳ و ۹ را می‌گیرد؛ ۲۷ دیگر از ۲۰ بیشتر است. پس **۳** تکرار.

۵. `0 + 1 + 3 + 4 + 5 = 13` (۲ رد می‌شود و با ۶ خارج می‌شویم).

۶. مقایسه‌ها: `10>7` (i=9)، `9>7` (i=8)، `8>7` (i=7)، `7>7` نادرست ولی کاهش باز هم انجام می‌شود، پس **i=6**.

۷. زوج‌های `j < i` با `i + j` فرد: `(1,0)`، `(2,1)`، `(3,0)`، `(3,2)`، `(4,1)`، `(4,3)`. یعنی **۶**.

۸. `-4 % 3 = -1` (علامت عدد اول)، پس `case -1` اجرا می‌شود و جواب **۳**.

۹. دورها: `(0,10)`، `(2,8)`، `(4,6)` شمرده می‌شوند و با `(6,4)` شرط نقض می‌شود. خروجی `3 6 4`.

۱۰. برای `i=1`: با j=1 رد و با j=2 خارج. برای `i=2`: فقط ۲۱. برای `i=3`: `31 + 32 = 63`. جمع **84**.